Universidad Nacional Autó
Facultad de Ciencias



Inteligencia Artificial

Laboratorio 2.3

Docente: *Dra. JessicaSarahiMéndezRincón*

Ayudante de laboratorio: *DiegoEduardoPeñaVillegas*

Alumna: *OsorioLopezClaudiaYarinet*

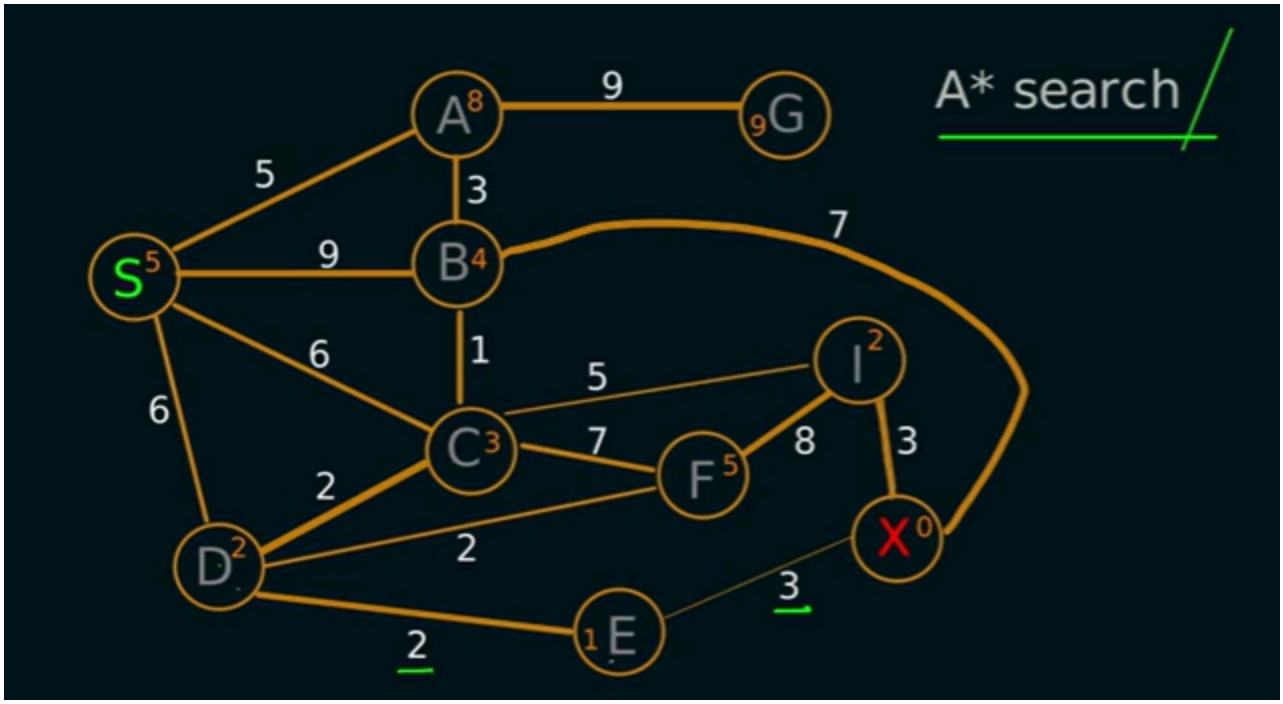
Fecha de realización: 14/09/2026

Introducción

Revisar el siguiente video para comprender el problema:
https://www.youtube.com/watch?v=yxN6yR_7yJM

Desarollo

El siguiente código ayudará mediante el algoritmo A* a encontrar el camino más optimo:




```
import heapq

def a_star(grafo, heuristicas, inicio, destino):
    # priority_queue guardará tuplas: (f_score, nodo, camino_actual, g_score)
    frontera = [(heuristicas[inicio], inicio, [inicio], 0)]
    visitados = set()

    while frontera:
        # Extraemos el nodo con el menor f(n) = g(n) + h(n)
        (f, nodo_actual, camino, g) = heapq.heappop(frontera)

        if nodo_actual == destino:
            return camino, g

        if nodo_actual not in visitados:
            visitados.add(nodo_actual)

            # Explorar vecinos
            for vecino, costo in grafo[nodo_actual].items():
                if vecino not in visitados:
                    nuevo_g = g + costo
                    nuevo_f = nuevo_g + heuristicas[vecino]
                    nuevo_camino = camino + [vecino]
                    heapq.heappush(frontera, (nuevo_f, vecino, nuevo_camino, nu

    return None, float('inf')

# 1. Definición del Grafo (según las conexiones de tu imagen)
grafo = {
    'S': {'A': 5, 'B': 9, 'D': 6},
```

```
    'A': {'S': 5, 'B': 3, 'G': 9},
    'B': {'S': 9, 'A': 3, 'C': 1},
    'C': {'B': 1, 'D': 2, 'F': 7, 'I': 5},
    'D': {'S': 6, 'C': 2, 'E': 2},
    'E': {'D': 2, 'X': 3},
    'F': {'C': 7, 'I': 8, 'X': 2},
    'I': {'C': 5, 'F': 8, 'X': 3},
    'G': {'A': 9},
    'X': {'E': 3, 'F': 2, 'I': 3}
}

# 2. Valores de la Heurística h(n) (los números pequeños arriba de cada letra)
heuristicas = {
    'S': 5, 'A': 8, 'B': 4, 'C': 3, 'D': 2,
    'E': 1, 'F': 5, 'I': 2, 'G': 9, 'X': 0
}

# Ejecución
camino_optimo, costo_total = a_star(grafo, heuristicas, 'S', 'X')

print(f"Camino encontrado: {' -> '.join(camino_optimo)}")
print(f"Costo total (f): {costo_total}")
```

```
Camino encontrado: S -> D -> E -> X
Costo total (f): 11
```

Otros ejemplos son como a continuación

```
pip install matplotlib
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-pack
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dis
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-pa
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/di
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/di
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.13/dist-pac
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-packa
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dis
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packag
```

```
import heapq

class Nodo:
    def __init__(self, estado, padre=None, costo=0, heuristica=0):
        self.estado = estado
        self.padre = padre
        self.costo = costo
```

```
        self.heuristica = heuristica

    def __lt__(self, otro):
        return (self.costo + self.heuristica) < (otro.costo + otro.heuristica)

def a_estrella(inicial, objetivo, heuristica):
    frontera = []
    heapq.heappush(frontera, Nodo(inicial, None, 0, heuristica(inicial, objetiv

    explorado = set()

    while frontera:
        actual = heapq.heappop(frontera)

        if actual.estado == objetivo:
            # Construir el camino desde el nodo objetivo hasta el inicial
            camino = []
            while actual:
                camino.append(actual.estado)
                actual = actual.padre
            return list(reversed(camino))

        explorado.add(actual.estado)
       #se inicializa un ciclo for para encontrar los vecinos cercanos
        for vecino in expandir(actual.estado):
            if vecino not in explorado:
                nuevo_costo = actual.costo + 1
                nuevo_nodo = Nodo(vecino, actual, nuevo_costo, heuristica(vecir
                heapq.heappush(frontera, nuevo_nodo)

    return None

def expandir(estado):
    # Esta función debe ser personalizada según el problema específico
    # Aquí hay un ejemplo simple para un tablero cuadrado de 3x3
    movimientos = [(0, 1), (1, 0), (0, -1), (-1, 0)]
    filas, columnas = 3, 3
    r, c = estado
    vecinos = [(r + dr, c + dc) for dr, dc in movimientos if 0 <= r + dr < fila
    return vecinos

def heuristica_euclidiana(posicion, objetivo):
    # Heurística euclidiana para la distancia entre dos puntos en un espacio 2D
    return ((posicion[0] - objetivo[0]) ** 2 + (posicion[1] - objetivo[1]) ** 2

# Ejemplo de uso
inicio = (0, 0)
meta = (2, 2)
ruta = a_estrella(inicio, meta, heuristica_euclidiana)

print("Camino encontrado:", ruta)
```

```
Camino encontrado: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2)]
```

Ahora mostrado como Nodos de un grafo:

```
pip install networkx
```

```
Requirement already satisfied: networkx in /usr/local/lib/python3.13/dist-packag
```

```
import heapq
import networkx as nx
import matplotlib.pyplot as plt

class Nodo:
    def __init__(self, estado, padre=None, costo=0, heuristica=0):
        self.estado = estado
        self.padre = padre
        self.costo = costo
        self.heuristica = heuristica

    def __lt__(self, otro):
        return (self.costo + self.heuristica) < (otro.costo + otro.heuristica)

def a_estrella(inicial, objetivo, heuristica):
    frontera = []
    heapq.heappush(frontera, Nodo(inicial, None, 0, heuristica(inicial, objetiv

    explorado = set()

    while frontera:
        actual = heapq.heappop(frontera)

        if actual.estado == objetivo:
            # Construir el camino desde el nodo objetivo hasta el inicial
            camino = []
            while actual:
                camino.append(actual.estado)
                actual = actual.padre
            return list(reversed(camino))

        explorado.add(actual.estado)

        for vecino in expandir(actual.estado):
            if vecino not in explorado:
                nuevo_costo = actual.costo + 1
                nuevo_nodo = Nodo(vecino, actual, nuevo_costo, heuristica(vecir
                heapq.heappush(frontera, nuevo_nodo)
```

```
    return None

def expandir(estado):
    # Esta función debe ser personalizada según el problema específico
    # Aquí hay un ejemplo simple para un tablero cuadrado de 3x3
    movimientos = [(0, 1), (1, 0), (0, -1), (-1, 0)]
    filas, columnas = 3, 3
    r, c = estado
    vecinos = [(r + dr, c + dc) for dr, dc in movimientos if 0 <= r + dr < fila
    return vecinos

def heuristica_euclidiana(posicion, objetivo):
    # Heurística euclidiana para la distancia entre dos puntos en un espacio 2D
    return ((posicion[0] - objetivo[0]) ** 2 + (posicion[1] - objetivo[1]) ** 2

# Ejemplo de uso
inicio = (0, 0)
meta = (2, 2)
ruta = a_estrella(inicio, meta, heuristica_euclidiana)

# Crear un grafo dirigido
G = nx.DiGraph()

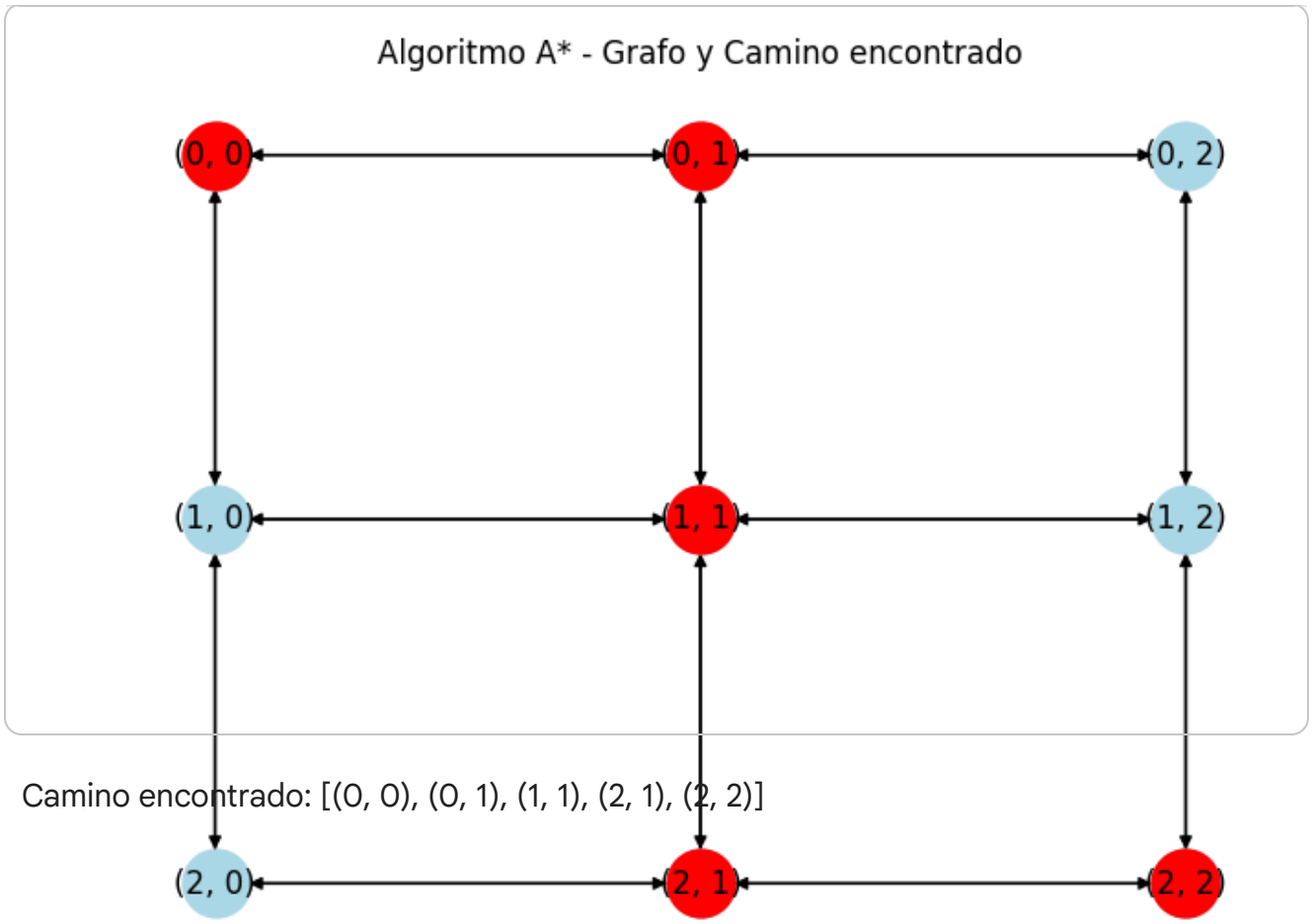
# Agregar nodos al grafo
for i in range(3):
    for j in range(3):
        G.add_node((i, j))

# Agregar arcos al grafo
for i in range(3):
    for j in range(3):
        vecinos = expandir((i, j))
        G.add_edges_from([((i, j), vecino) for vecino in vecinos])

# Visualizar el grafo
pos = {(i, j): (j, -i) for i in range(3) for j in range(3)}  # Ajustar la posic
nx.draw(G, pos, with_labels=True, node_size=700, node_color="lightblue")

# Resaltar el camino encontrado
nx.draw_networkx_nodes(G, pos, nodelist=ruta, node_color="red", node_size=700)

plt.title('Algoritmo A* - Grafo y Camino encontrado')
plt.show()
```



Camino encontrado: [(0, 0), (0, 1), (1, 1), (2, 1), (2, 2)]